

DEVOPS MAJOR PROJECT

**End-to-End Kubernetes CI/CD Pipeline using GitHub,
Jenkins, Docker & Kubernetes**

**M VINEETH KUMAR
JUNE BATCH -2026
3:30PM TO 5:00PM**

INDEX PAGE

- **INTRODUCTION**
- **TOOLS**
- **ARCHITECTURE**
- **WORKFLOW**
- **EXPLANATION**
- **SCRIPT TO RUN**
- **CONCLUTION**
- **OUTPUT**

1. INTRODUCTION

- This project is a complete CI/CD pipeline that automates the process of building, testing, and deploying applications.
- GitHub is used as the source code repository, while Jenkins automates the build and deployment workflow.
- Docker is used to containerize the application, ensuring consistency across different environments.
- The Docker image is then pushed to a container registry and deployed to a Kubernetes cluster.
- Kubernetes automates application deployment, scaling, and management, ensuring high availability and reliability of containerized applications.

2. TOOLS

- Git-hub – Source Code Management.
- Git - Version Control System
- Jenkins – Automation testing CI/CD.
- Kubernetes – Manage the service with deployment and scaling.
- Docker - Containerization Platform
- Docker-hub – Docker Image Repository
- Ec2 - Hosting Jenkins & Kubernetes.
- Kubectl - Kubernetes Command Line Tool.

3. ARCHITECTURE

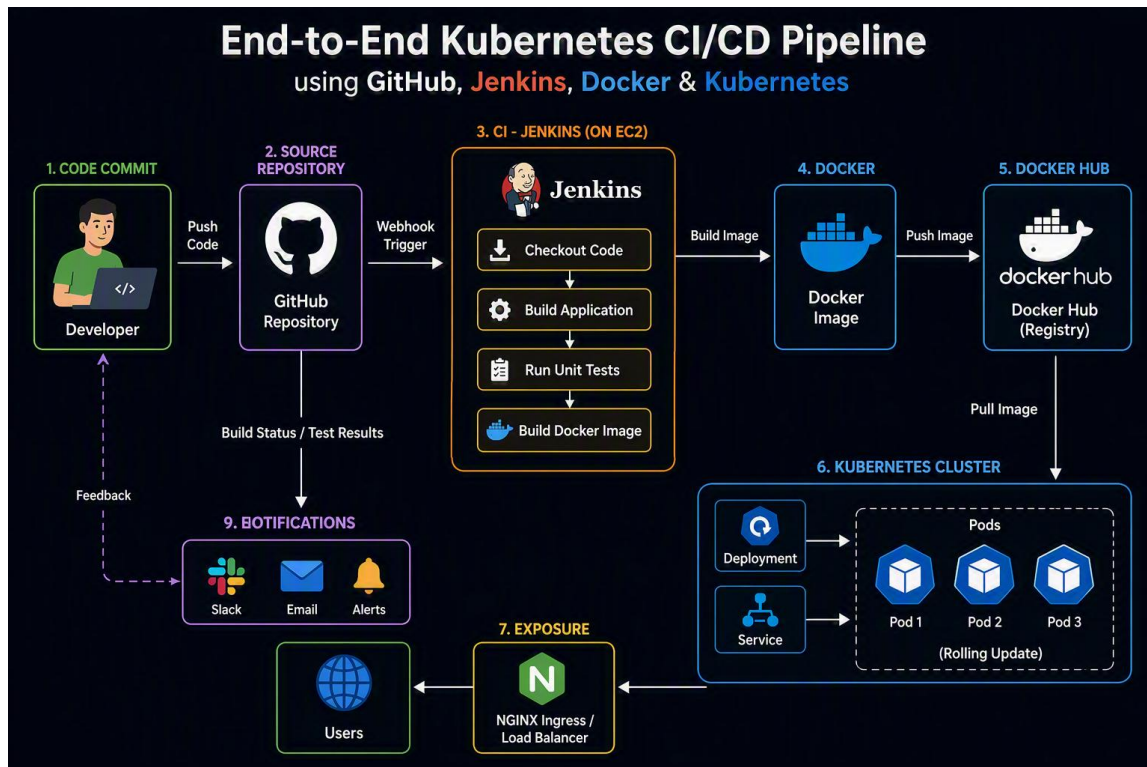
- Frontend - EC2 server(ubuntu)
 - Instance type c7i-flex.large with 30gb storage
- Jenkins – create a Jenkins user account for CI/CD code to test, build and deploy.
- Docker Hub - Create a docker-hub user account for push the manual image.

- Git-Hub – Create a git-hub repository to push & pull the code to reflect in CI/CD
- Kubernetes – Create deployment and service file to manage configure your service.

4. WORKFLOW:

- Create a ec2 instance
 - To launch front end nginx web server
- Install Jenkins
 - Create Jenkinsfile to write the pipeline syntax.
 - Jenkins pulls the latest code.
 - Jenkins builds the application.
- Install docker in your ec2 server.
 - Create Dockerfile to create a manual images or service.
 - Docker image is pushed to Docker Hub
- Install github in your ec2 server.
 - Developer pushes code to **GitHub**.
 - Push the code to CI/CD to trigger the build job in Jenkins
- Install Kubernetes and minikube in your ec2 server.
 - Create Deployment and Service .yaml file manage the configure the container.
 - Kubernetes pulls the latest image.

5. DIAGRAM:



6. WORKFLOW STEPS:

- Create an EC2 server

```
aws
[Alt+S]
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-1011-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Aug 24 06:41:04 UTC 2026

System load:  0.23           Temperature:   -273.1 C
Usage of /:   33.0% of 28.02GB Processes:    163
Memory usage: 25%           Users logged in: 0
Swap usage:   0%            IPv4 address for enp39s0: 172.31.47.52

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.
   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.
17 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sat Aug 22 07:04:27 2026 from 13.233.177.4
ubuntu@ip-172-31-47-52:~$
```

i-Oe32fe319093c9caa (Dev_project)
PublicIPs: 13.201.87.82 PrivateIPs: 172.31.47.52

allow the port no in sg which we gonna to use.

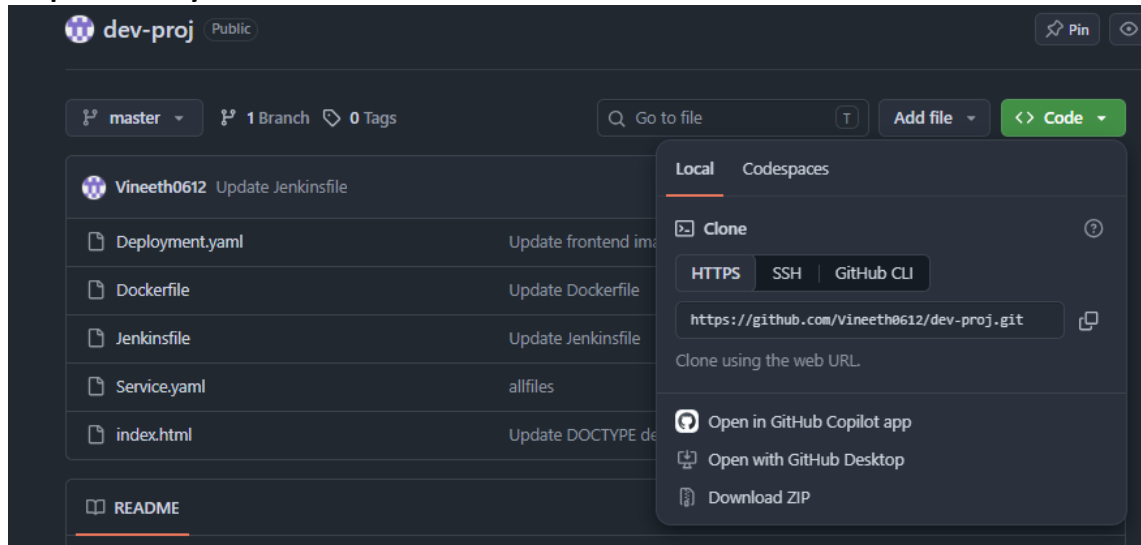
- Port no to allow:
 - Jenkins port: 8080
 - Nginx: 80
 - Nord port no: 30080.

▼	Security group rule ID ▼	IP version ▼	Type ▼	Protocol ▼	Port range ▼	Source ▼
	sgr-0df5490b7835afa8a	IPv4	Custom TCP	TCP	8282	0.0.0.0/0
	sgr-0e0ed9aae2f1fee8f	IPv4	SSH	TCP	22	0.0.0.0/0
	sgr-05bd7b08abd1cc8d2	IPv4	Custom TCP	TCP	8080	0.0.0.0/0
	sgr-082cfddd7be902d9c	IPv4	Custom TCP	TCP	8181	0.0.0.0/0
	sgr-0080cc206657f6962	IPv4	Custom TCP	TCP	30090	0.0.0.0/0
	sgr-0c5142265f62fc869	IPv4	All ICMP - IPv4	ICMP	All	0.0.0.0/0
	sgr-0c471ddc9b998d576	IPv4	HTTPS	TCP	443	0.0.0.0/0
	sgr-05b3962811cae2f31	–	All traffic	All	All	sg-09abf4067a29da03b...
	sgr-048eafe66ff5697c	IPv4	Custom TCP	TCP	8383	0.0.0.0/0
	sgr-05fb423d1ac0b0456	IPv4	Custom TCP	TCP	30080	0.0.0.0/0
	sgr-0265f8e17ac1820c3	IPv4	HTTP	TCP	80	0.0.0.0/0

- Create a directory and create a Jenkinsfile, Dockerfile index.html Deployment.yaml and Service.yaml file.

```
ubuntu@ip-172-31-47-52:~/Devops$ ls
Deployment.yaml  Dockerfile  Jenkinsfile  Service.yaml  index.html
ubuntu@ip-172-31-47-52:~/Devops$
```

- Create a git hub repository to push & pull the code to your repository.



- Create a Jenkins user account and create a pipeline project and select the scripted pipeline from SCM.
 - Create credential for github.

Sign in to Jenkins

Username

Password

☐ Keep me signed in

New Item

Enter an item name

Select an item type

☒ Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

- Create a docker-hub account to push & pull the services.

TAG		
vineeth		
Last pushed 3 days by vineeth0612		
☐	Digest	OS/ARCH
	24534465a6e1	linux/amd64
		Last pull
		less than 1 day

- Install the minikube

```
ubuntu@ip-172-31-47-52:~/.kube$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

ubuntu@ip-172-31-47-52:~/.kube$ kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
dev-proj-749dc5546-jfs85            0/1     ImagePullBackOff    0           2d
dev-proj-749dc5546-t77kc            0/1     ImagePullBackOff    0           2d
ubuntu@ip-172-31-47-52:~/.kube$ kubectl get nodes
NAME      STATUS   ROLES    AGE   VERSION
minikube  Ready    control-plane  5d16h  v1.35.1
ubuntu@ip-172-31-47-52:~/.kube$
```

- create a files in your server.

```
ubuntu@ip-172-31-47-52:~/Devops$ ls
Deployment.yaml  Dockerfile  Jenkinsfile  Service.yaml  index.html
ubuntu@ip-172-31-47-52:~/Devops$
```

7.SCRIPT TO RUN

Jenkins syntax:

```
Jenkinsfile.js X
Jenkinsfile.js
1 pipeline {
2   agent any
3   environment {
4     DOCKER_IMAGE = "vineeth0612/proj:latest"
5     DOCKERHUB_CREDENTIALS = credentials('dockerhub-creds')
6   }
7   stages {
8     stage('Checkout') {
9       steps {
10         git branch: 'master',
11            url: 'https://github.com/Vineeth0612/Devops_project.git'
12       }
13     }
14     stage('Build Docker Image') {
15       steps {
16         sh 'docker build -t $DOCKER_IMAGE:$BUILD_NUMBER .'
17       }
18     }
19     stage('Login Docker Hub') {
20       steps {
21         withCredentials([
22           usernamePassword(
23             credentialsId: 'Vineeth0612/***** (Git_cred)',
24             usernameVariable: 'Vineeth0612',
25             passwordVariable: 'Vineeth@7358'
26           )
27         ]) {
28           sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
29         }
30       }
31     }
32     stage('Push Docker Image') {
33       steps {
34         sh 'docker push $DOCKER_IMAGE:$BUILD_NUMBER'
35         sh 'docker tag $DOCKER_IMAGE:$BUILD_NUMBER $DOCKER_IMAGE:latest'
36         sh 'docker push $DOCKER_IMAGE:latest'
37       }
38     }
39     stage('Deploy to Kubernetes') {
40       steps {
41         sh 'kubectl apply -f k8s/deployment.yaml'
42         sh 'kubectl apply -f k8s/service.yaml'
43       }
44     }
45     stage('Verify Deployment') {
46       steps {
47         sh 'kubectl get pods'
48         sh 'kubectl get services'
49       }
50     }
51   }
52 }
```

Dockerfile:

```
Dockerfile.txt X
Dockerfile.txt > ...
1 FROM nginx:latest
2
3 WORKDIR /app
4
5 COPY index.html usr/share/nginx/html/
6
7 COPY . .
8
9 EXPOSE 80
10
11 CMD ["nginx" , "index.html"]
```

Deployment.yaml

```
! Deployment.yaml X
! Deployment.yaml
1 apiVersion: apps/v1
2 kind: Deployment
3
4 metadata:
5   name: dev-proj
6
7 spec:
8   replicas: 2
9
10  selector:
11    matchLabels:
12      app: dev-proj
13
14  template:
15    metadata:
16      labels:
17        app: dev-proj
18
19    spec:
20      containers:
21        - name: frontend
22          image: vineeth0612/k8s-deploy:nginx
23
24      ports:
25        - containerPort: 80
```

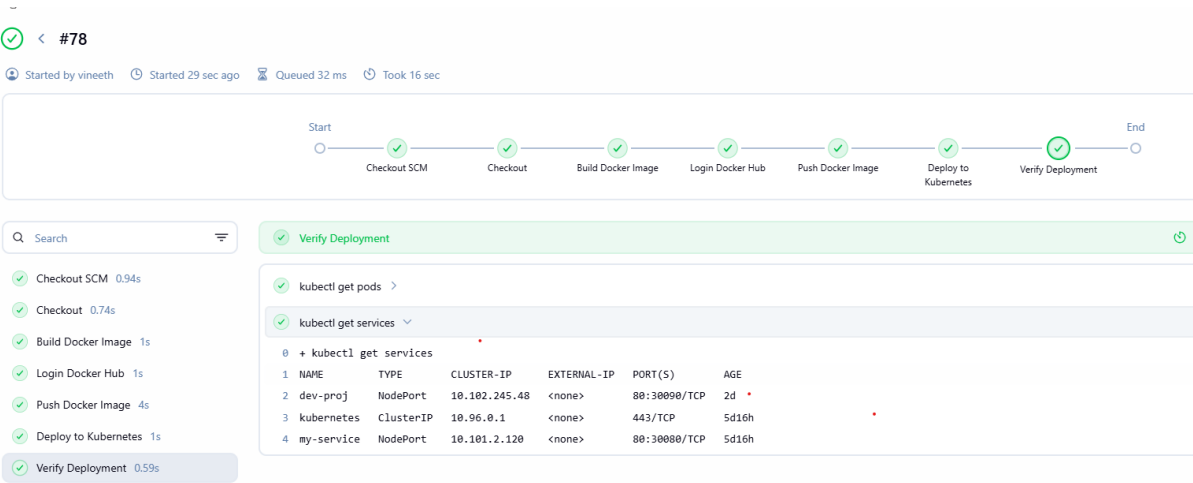
Service.yaml

```
! Service.yaml X
! Service.yaml
1  apiVersion: v1
2  kind: Service
3
4  metadata:
5    name: dev-proj
6
7  spec:
8    type: NodePort
9
10   selector:
11     app: nginx
12
13   ports:
14     - port: 80
15       targetPort: 80
16       nodePort: 30080
```

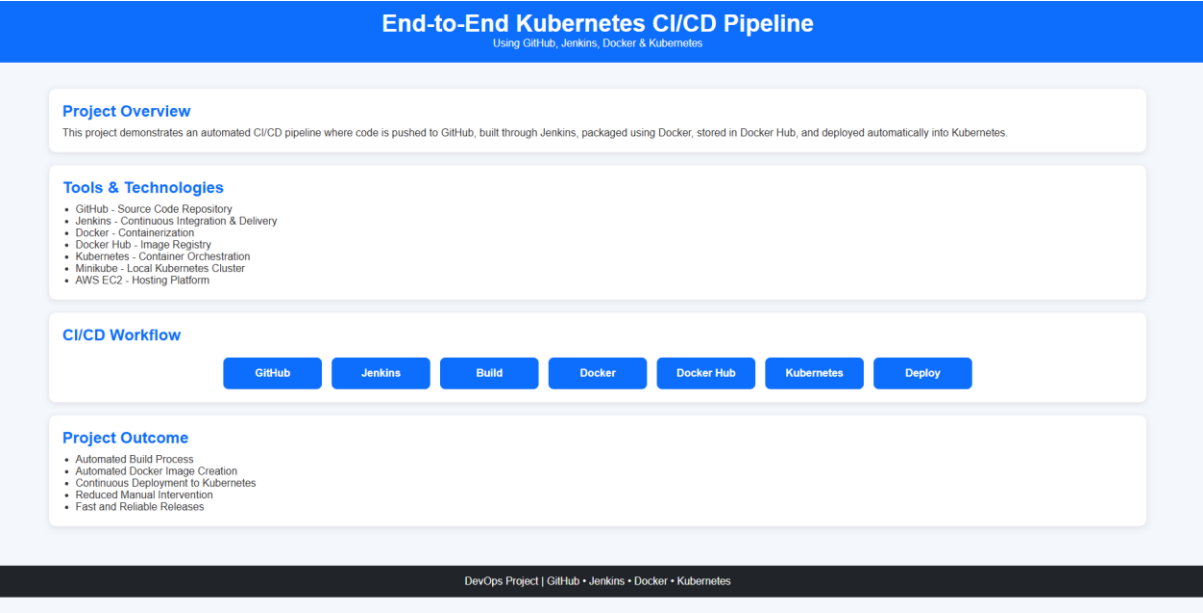
7. OUTPUT:

- Automated build and deployment process.
- Docker images stored in Docker Hub.
- Kubernetes deployment created successfully.
- Application running inside Kubernetes Pods.
- Reduced manual deployment effort.
- Faster and reliable software delivery

CI/CD OUTPUT



WEBSERVER OUTPUT:



8. CONCLUTION

The End-to-End Kubernetes CI/CD Pipeline integrates GitHub, Jenkins, Docker, and Kubernetes to automate the complete software delivery lifecycle. By implementing this pipeline, organizations can achieve faster deployments, improved reliability, enhanced scalability, and consistent application delivery. This project demonstrates modern DevOps practices and provides a strong foundation for enterprise-level cloud-native deployments.